TIẾT 2: TIẾN HÓA NGÔN NGỮ, CÔNG NGHỆ ĐỐI TƯỢNG & KIẾN TRÚC .NET

1.5 Các Thế hệ Ngôn ngữ Lập trình (Language Evolution)

1. Ngôn ngữ máy (Machine Language)

- Là ngôn ngữ **duy nhất** phần cứng CPU có thể hiểu và thực thi trực tiếp.
- Cấu thành từ các chuỗi số nhị phân `0` và `1`.
- Phụ thuộc chặt chẽ vào kiến trúc phần cứng máy tính (Machine-dependent).
- Cực kỳ khó viết, dễ sai sót và không thể bảo trì bởi con người.

2. Hợp ngữ (Assembly Language)

- Sử dụng các từ gợi nhớ tiếng Anh (Mnemonics) viết tắt: `LOAD`, `ADD`, `STORE`.
- Cần chương trình **Assembler** để dịch hợp ngữ sang ngôn ngữ máy.
- Vẫn phụ thuộc vào dòng vi xử lý cụ thể, tốc độ viết code vẫn rất chậm.

3. Ngôn ngữ bậc cao (High-Level Language): C, C++, Java, C#, Python

Cú pháp gần với ngôn ngữ tự nhiên tiếng Anh và ký hiệu toán học; độc lập với phần cứng.

Biên dịch (Compiler) vs Thông dịch (Interpreter)

Trình biên dịch (Compiler)

- Dịch **toàn bộ** mã nguồn bậc cao thành tệp mã máy (Machine code) trước khi thực thi.
- Quá trình biên dịch tốn thời gian ban đầu nhưng chương trình chạy **cực kỳ nhanh**.
- Báo lỗi cú pháp toàn diện trước khi cho phép chạy chương trình.
- *Đại diện:* C, C++, Rust, Go.

Trình thông dịch (Interpreter)

- Đọc và dịch **từng dòng lệnh một**, chạy trực tiếp lệnh đó rồi mới sang dòng kế tiếp.
- Chạy ngay không cần chờ biên dịch, nhưng tốc độ thực thi **chậm hơn đáng kể**.
- Lỗi ở dòng nào chỉ phát hiện khi máy tính chạy đến dòng đó.
- *Đại diện:* Python, JavaScript, Ruby, PHP.

Cơ chế Lai (Hybrid Approach) trong C# / Java:

Biên dịch mã nguồn thành **Mã trung gian (MSIL / Bytecode)**, sau đó dùng **JIT Compiler** trong máy ảo để dịch sang mã máy siêu tốc khi ứng dụng khởi chạy!

1.6 Công nghệ Đối tượng (Object Technology)

Tư duy Hướng đối tượng trong Lập trình:

- Trong thế giới thực, mọi vật xung quanh chúng ta đều là các **Đối tượng (Objects)**: ô tô, điện thoại, con người, tài khoản.
- Mỗi đối tượng đều có hai thành phần:
  1. **Thuộc tính (Attributes / Properties):** Màu sắc, kích thước, số dư tài khoản.
  2. **Hành vi (Behaviors / Methods):** Chạy, dừng, tăng tốc, rút tiền, chuyển khoản.

Mục tiêu Tối thượng của OOP

Tái sử dụng Phần mềm (Software Reuse)

Thay vì tự gõ lại mọi dòng code từ đầu, ta sử dụng các lớp đối tượng có sẵn đạt chuẩn chất lượng cao, giúp tăng năng suất gấp nhiều lần.

Phân biệt Lớp (Class) và Đối tượng (Object)

Lớp (Class) = Bản vẽ thiết kế

- Là khuôn mẫu (Blueprint / Template) định nghĩa cấu trúc dữ liệu và hành vi chung.
- *Ví dụ:* Bản vẽ thiết kế kỹ thuật ô tô trên giấy: chỉ định xe có 4 bánh, vô lăng, màu sơn, động cơ.
- Bản vẽ **không thể tự nổ máy hay chạy trên đường**!

Đối tượng (Object) = Thực thể cụ thể

- Là một thể hiện cụ thể (Instance) được chế tạo từ bản vẽ lớp.
- *Ví dụ:* Chiếc xe VinFast màu đỏ biển số `43A-12345` thực sự lăn bánh ngoài đời.
- Từ 1 Class có thể tạo ra **vô số Objects** với các giá trị thuộc tính riêng biệt!

Cú pháp khởi tạo trong C#:

```
Car myCar = new Car("VinFast VF8", "Do"); // Toán tử new sinh đối tượng trong RAM
```

Các Trụ cột Công nghệ Đối tượng trong C#

1. Đóng gói (Encapsulation)

- Gom nhóm dữ liệu và phương thức xử lý vào trong một đơn vị Class duy nhất.
- **Che giấu thông tin (Information Hiding):** Người lái xe chỉ cần đạp chân ga để tăng tốc, không cần biết chi tiết quá trình bơm nhiên liệu vào buồng đốt!
- Bảo vệ dữ liệu nội bộ không bị can thiệp trái phép từ bên ngoài.

2. Kế thừa (Inheritance)

- Cho phép tạo ra một Class mới dựa trên một Class đã có sẵn.
- Lớp con thừa hưởng toàn bộ thuộc tính và hành vi của lớp cha, đồng thời có thể bổ sung tính năng riêng.
- *Ví dụ:* `XeDien` kế thừa từ `OTo`, bổ sung thêm thuộc tính `DungLuongPin`.

3. Đa hình (Polymorphism):

Cho phép các đối tượng thuộc các lớp khác nhau phản ứng với cùng một thông điệp theo những cách riêng biệt (sẽ học sâu ở Chương 8 & 9).

1.7 Internet và World Wide Web

Các cột mốc tiến hóa mạng thông tin toàn cầu:

- **ARPANET (1969):** Tiền thân của Internet, do Bộ Quốc phòng Mỹ tài trợ, tiên phong kỹ thuật chuyển mạch gói (Packet Switching).
- **Giao thức TCP/IP:** Bộ giao thức mạng nền tảng kết nối các mạng máy tính khác biệt trên thế giới.
- **World Wide Web (1989):** Tim Berners-Lee phát minh siêu văn bản (HTML) và giao thức HTTP, biến Internet thành không gian chia sẻ tài liệu toàn cầu.
- **Web 2.0 (Thập niên 2000):** Chuyển từ web đọc tĩnh sang web tương tác người dùng (Facebook, YouTube, Wikipedia, Cloud SaaS).
- **Điện toán đám mây (Cloud Computing - Microsoft Azure):** Thuê năng lực tính toán và lưu trữ quy mô toàn cầu theo nhu cầu thực tế.
- **Internet of Things (IoT):** Hàng tỷ thiết bị thông minh (đồng hồ, TV, cảm biến xe) kết nối và trao đổi dữ liệu qua Internet.

1.8 Ngôn ngữ C#: Lịch sử Hình thành & Chuẩn hóa

Lịch sử ra đời:

- Được Microsoft công bố vào năm 2000, đi cùng sáng kiến .NET.
- Người đứng đầu nhóm thiết kế: **Anders Hejlsberg** (Kỹ sư trưởng huyền thoại).
- C# được chuẩn hóa quốc tế bởi **ECMA (ECMA-334)** và **ISO/IEC (ISO/IEC 23270)**.

Nguồn gốc tên gọi C#:

Dấu thăng (#) mượn từ âm nhạc (thăng tiến), đồng thời ghép từ 4 dấu cộng (++ của C++ xếp chồng lên nhau thành #), biểu thị sự kế thừa và nâng cấp toàn diện!

Đặc trưng Cốt lõi của C#

- Hướng đối tượng 100% (Pure OOP)
- Định kiểu mạnh mẽ & An toàn (Type-safe)
- Thu gom rác tự động (Garbage Collection)
- Hỗ trợ lập trình trực quan (Visual)
- Hỗ trợ xử lý bất đồng bộ (`async/await`)
- Đa nền tảng (Cross-platform)

Các Phong cách Lập trình Đa dạng trong C#

1. Lập trình Hướng sự kiện (Event-Driven)

- Ứng dụng phản hồi lại các sự kiện (Events) do người dùng kích hoạt: Click chuột, gõ phím, di chuột.
- Cốt lõi của các ứng dụng GUI hiện đại.

2. Lập trình Trực quan (Visual Programming)

- Kéo - thả các điều khiển (Button, TextBox, Grid) trực tiếp trên Form Designer thay vì viết code giao diện thủ công.

3. Lập trình Tổng quát (Generic Programming)

- Viết thuật toán và cấu trúc dữ liệu tổng quát cho mọi kiểu dữ liệu mà vẫn đảm bảo Type-Safety (ví dụ `List<T>`).

4. Lập trình Bất đồng bộ (Async / Await)

- Thực thi các tác vụ tốn thời gian (I/O, gọi API mạng, truy vấn CSDL) mà **không làm đơ / treo giao diện** người dùng.

1.9 Nền tảng Microsoft .NET: Tầm nhìn Kiến trúc

Sáng kiến Microsoft .NET (Ra mắt năm 2000):

Một nền tảng phần mềm toàn diện cho phép phát triển, triển khai và thực thi ứng dụng dễ dàng trên Internet và mạng nội bộ.

1. Thư viện Lớp (.NET FCL)

- Hàng chục nghìn lớp có sẵn được tối ưu hóa hiệu năng cao.
- Cung cấp giải pháp cho: Thao tác tệp (File I/O), Mạng (Networking), CSDL (ADO.NET, Entity Framework), Bảo mật, Giao diện (WinForms, WPF).
- Tái sử dụng tối đa, tiết kiệm 70% thời gian gõ code.

2. Máy ảo CLR (Common Language Runtime)

- Trái tim vận hành của toàn bộ hệ sinh thái .NET.
- Quản lý toàn bộ vòng đời thực thi ứng dụng.
- Ẩn giấu phần cứng và hệ điều hành bên dưới đối với lập trình viên.
- Cung cấp các dịch vụ quản lý tự động: Cấp phát bộ nhớ, thu gom rác, kiểm tra bảo mật.

Khái niệm Managed Code (Mã được Quản lý)

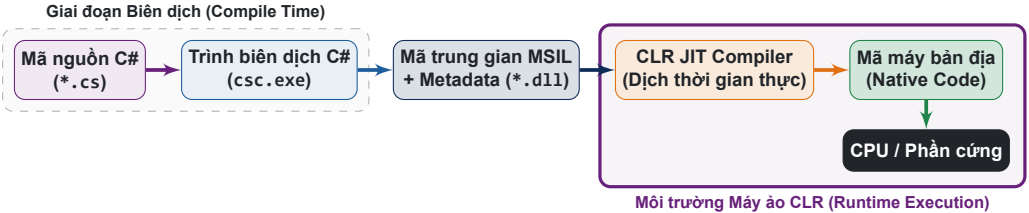
Managed Code (Mã được quản lý)

- Là mã nguồn được biên dịch ra mã trung gian MSIL và **chạy dưới sự giám sát hoàn toàn của CLR**.
- C#, VB.NET, F# mặc định luôn là Managed Code.
- **Lợi ích to lớn:**
  - Không lo rò rỉ bộ nhớ (Memory Leaks).
  - Ngăn chặn lỗi ghi đè vùng nhớ (Buffer Overflow).
  - Bắt lỗi ngoại lệ (Exceptions) an toàn giữa các ngôn ngữ khác nhau.

Unmanaged Code (Mã không quản lý)

- Được biên dịch trực tiếp sang mã máy nhị phân của CPU (ví dụ C, C++ truyền thống).
- Không có máy ảo CLR quản lý.
- Lập trình viên phải tự quản lý từng byte bộ nhớ (`malloc`, `free`, con trỏ).
- Nếu có lỗi con trỏ có thể làm sập toàn bộ hệ điều hành (Crash / BSoD).

Kiến trúc Thực thi .NET: Mã nguồn → MSIL → CLR JIT



Vai trò của MSIL (Microsoft Intermediate Language):

MSIL không phải mã máy của Intel hay ARM. Nhờ đó, tệp `*.dll` có thể mang sang bất kỳ máy tính nào chạy Windows, Linux hay macOS có cài đặt CLR để thực thi mượt mà!

1.9.3 Độc lập Nền tảng (Platform Independence)

Triết lý "Write Once, Run Anywhere":

- Lập trình viên chỉ cần viết và biên dịch code một lần thành MSIL.
- Bất kỳ hệ điều hành nào có máy ảo CLR tương thích đều có thể thực thi chương trình mà **không cần sửa hay biên dịch lại mã nguồn**.
- Tiết kiệm hàng triệu USD chi phí chuyển đổi (porting) phần mềm cho doanh nghiệp.

Sự tiến hóa của .NET

- **.NET Framework (2002 - 2019):** Ban đầu chủ yếu tối ưu hóa cho hệ điều hành Windows.
- **.NET Core (2016):** Viết lại hoàn toàn với mã nguồn mở, hoạt động siêu tốc trên Windows, Linux, macOS.
- **.NET 5 → 8 → 9 (Hiện đại):** Hợp nhất toàn bộ hệ sinh thái .NET thành một nền tảng duy nhất đẳng cấp thế giới.

1.9.4 Khả năng Tương tác Đa ngôn ngữ (Language Interoperability)

Tự do lựa chọn Ngôn ngữ:

- Nền tảng .NET không bắt buộc lập trình viên phải dùng chung một ngôn ngữ.
- Lập trình viên có thể viết Module A bằng **C#**, Module B bằng **Visual Basic**, và Module C bằng **F#**.
- Tất cả đều được biên dịch về cùng một chuẩn **mã trung gian MSIL**!

Sức mạnh Tích hợp Không biên giới:

- Một lớp viết bằng C# có thể **kế thừa trực tiếp** một lớp viết bằng Visual Basic.
- Một phương thức viết bằng C# có thể bắt ngoại lệ (Exception) do mã F# ném ra.
- Toàn bộ thư viện .NET Framework Class Library (FCL) được chia sẻ bình đẳng cho tất cả các ngôn ngữ .NET.

[CHECKPOINT 2] Kiểm tra Nhanh Kiến thức Tiết 2

Câu hỏi 1: Mã nguồn C# khi biên dịch lần đầu sẽ tạo ra loại mã nào?

A. Mã máy nhị phân của CPU Intel x86/x64
B. **Mã trung gian Microsoft Intermediate Language (MSIL / CIL)** *[ĐÁP ÁN ĐÚNG]*
C. Mã hợp ngữ Assembly
D. Mã thông dịch Python bytecode

Câu hỏi 2: Thành phần nào trong CLR dịch MSIL sang mã máy khi ứng dụng chạy?

A. Trình soạn thảo Visual Studio
B. Trình liên kết (Linker)
C. **Trình biên dịch Just-In-Time (JIT Compiler)** *[ĐÁP ÁN ĐÚNG]*
D. Trình dọn rác (Garbage Collector)

[HANDS-ON LAB 2] Trải nghiệm Chu trình Biên dịch .NET qua CLI

Thực hành Biên dịch & Chạy ứng dụng C# Console (10 Phút):

Mở PowerShell hoặc Command Prompt tại máy tính sinh viên và thực hiện 3 lệnh sau:

```
# 1. Kiem tra phien ban .NET SDK da cai dat tren may tinh
C:\> dotnet --version
8.0.204

# 2. Tao mot du an C# Console moi mang ten "HelloIT26M"
C:\> dotnet new console -o HelloIT26M
The template "Console App" was created successfully.

# 3. Di chuyen vao thu muc du an va chay chuong trinh
C:\> cd HelloIT26M
C:\HelloIT26M> dotnet run
Hello, World!
```

Quan sát quan trọng:

Lệnh `dotnet run` tự động gọi trình biên dịch sinh ra tệp `HelloIT26M.dll` (chứa mã MSIL) trong thư mục `bin/Debug/net8.0/`, sau đó kích hoạt CLR để chạy!